

Introduction à Git

Formation Robotech

Timothée

`https://github.com/Robotech-Lillois`

28 septembre 2026



 Ce support est mis à disposition sous licence **CC BY-NC-SA 4.0**

Sommaire

- 1 Introduction
- 2 Philosophie
- 3 Les bases
- 4 Branches
- 5 Setup
- 6 Exercice
- 7 Commandes avancées
- 8 Conclusion

Qu'est-ce que Git?

- **Un gestionnaire de versions :**
 - Conçu en 2005 par *Linus Torvalds* pour gérer du code
 - Sauvegarde toutes les modifications
- **Un outil de travail collaboratif :**
 - Facilite le partage de code
 - Permet de travailler en même temps sur un même projet
- **Un outil efficace et rapide :**
 - Conçu initialement pour le kernel Linux
 - Fait pour être utilisé par de grands projets

Pourquoi Git?

- **Des dizaines de versions dans un seul fichier** : `projet_final.odt`, `projet_v2_final_final5.odt`, ... ⇒ **projet.odt**
- **Droit à l'erreur** : Possibilité d'inspecter ou d'annuler n'importe quelle modification passée
- **Travail collaboratif plus simple** : Plusieurs personnes peuvent coder simultanément sans écraser le code des autres
- **Branches indépendantes** : Tester des fonctionnalités sans casser les autres versions
- **Standard universel** du monde pro ET de la recherche.

Git vs GitHub

outil local VS service d'hébergement

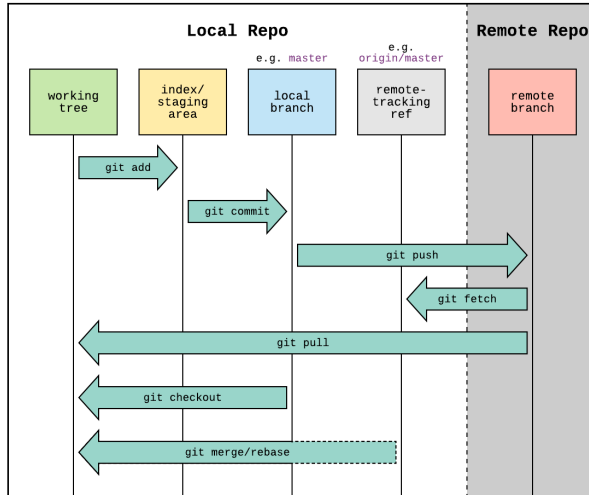
Git (outil local)

- Logiciel en ligne de commande (CLI)
- 100% autonome sans accès Internet
- open-source
- Crée les commits, gère les branches et les fusions

GitHub / GitLab / Gitea (Plateforme Web)

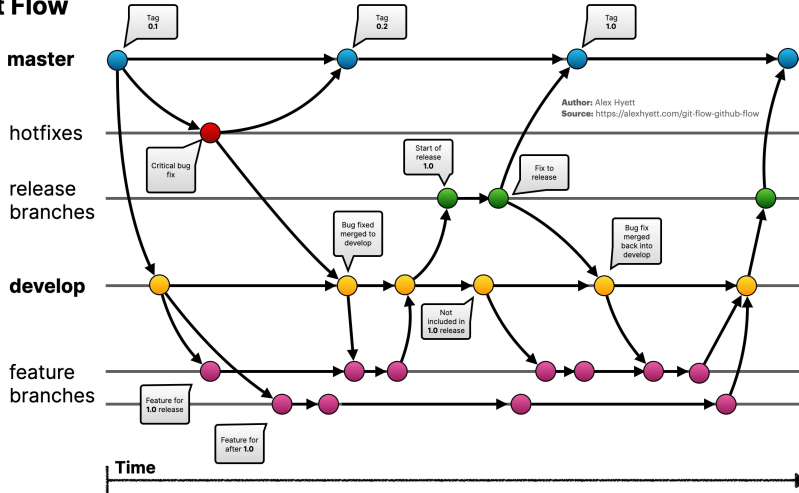
- Serveur d'hébergement distant
- Interface graphique et exploration web
- Autres outils : Pull Requests, ...

Le Workflow Git



Git Flow

Git Flow



Commandes de base

initialiser - préparer - enregistrer

- 1 `git init` : Initialise un dépôt local dans le dossier courant (crée `.git/`).

```
git init
```

- 2 `git add` : Ajoute des fichiers modifiés à l'index (*staging area*).

```
git add main.py           # Ajoute un fichier précis
git add .                  # Prépare l'ensemble des fichiers modifiés
```

- 3 `git commit` : Enregistre le commit avec un message **explicite!**

```
git commit -m "feature: ajout de la fonction add_vectors dans math.h"
```

Commandes de base

Connaître l'état du projet

- 1 `git status` : Présente l'état des fichiers (modifiés, suivis, indexés).

```
git status
```

- 2 `git diff` : Affiche les écarts exacts de code ligne par ligne.

```
git diff                # Modifications non indexées
git diff --staged        # Modifications prêtes pour le prochain commit
```

- 3 `git log` : Visualise l'historique chronologique des commits.

```
git log --oneline --graph --all
```

Commandes de base

Synchronisation distante

- 1 `git clone` : Télécharge une copie intégrale d'un dépôt distant existant.

```
git clone https://github.com/Robotech-Lillois/Chessboard.git
```

- 2 `git remote` : Ajoute un repo distant comme source

```
git remote add origin https://github.com/Robotech-Lillois/Formation
```

- 3 `git pull` : Récupère et fusionne les commits distants sur votre branche locale.

```
git pull origin main
```

- 4 `git push` : Publie vos commits locaux validés vers le serveur distant.

```
git push origin main
```

Le fichier .gitignore

Exclure certains fichiers

- Fichier placé à la racine du dépôt indiquant les motifs à ne jamais indexer.
- Indispensable pour garder un dépôt clair et ne pas faire fuiter certaines infos.

Exemple de fichier .gitignore

```
# Fichiers sensibles (JAMAIS sur un dépôt Git !)  
mot-de-passe.txt  
  
# Dépendances et fichiers générés  
__pycache__/  
*.o  
*.pdf  
  
# Fichiers temporaires des systèmes d'exploitation  
.DS_Store  
Thumbs.db
```

Le README.md

Un fichier à ne pas négliger

Pourquoi est-il indispensable?

- **1ère impression** : 1er fichier lu et affiché par défaut sur GitHub / GitLab
- **Autonomie** : permet à un arrivant d'installer et tester le projet sans vous solliciter
- **Format Markdown** : simple en texte brut, riche à l'affichage web

Règle d'or

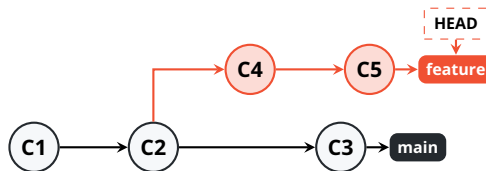
Un code sans README est inutilisable pour les autres

Structure minimale en 5 points

- 1 **Titre & Pitch** : nom du projet et rôle en 1 phrase.
- 2 **Installation** : dépendances et commandes de setup.
- 3 **Usage** : exemple minimal pour lancer le code.
- 4 **Contribution** : consignes pour soumettre une PR.
- 5 **Licence** : conditions légales d'utilisation (MIT, GPL...).

Les Branches

Ne pas travailler sur le main !



Commandes principales

```
git switch -c feature-strophe  # Crée et bascule sur la branche feature
git branch -v                  # Liste toutes les branches locales
git switch main                 # Revient sur la branche principale
```

Le Merge

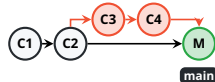
Comparaison des options `-ff`, `-no-ff` et `-ff-only`

`-ff` (Défaut)



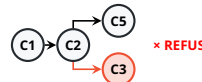
- Avance simple du pointeur
- Zéro commit créé si ligne droite
- Divergence $\Rightarrow$ 3-way merge auto
- Risque : crée des commits de fusion involontaires lors d'un pull

`-no-ff`



- Commit de fusion **M** forcé
- Isole et groupe la fonctionnalité
- Facilite l'annulation complète d'une feature (revert en 1 commit)
- Recommandé pour les *Pull Requests*

`-ff-only`

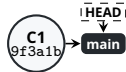


- Avance rapide obligatoire
- Divergence $\Rightarrow$ Arrêt net (*abort*)
- Zéro fichier pollué par un conflit
- Impose un rebase préalable (garantit un historique 100% plat)

Note : `ff` = *fast forward*

Exemple de conflit (1/7)

Init et 1er commit



Commandes Bash

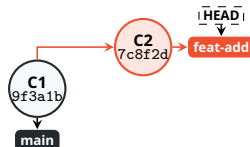
```
git init
echo "#define MATH_H" > math.h
git add math.h
git commit -m "init: base lib math"
```

Fichier `math.h` (commit C1)

```
#ifndef MATH_H
#define MATH_H
// Fonctions maths a venir...
#endif
```

Exemple de conflit (2/7)

Création de la branche `feat-add` et ajout de l'addition



Commandes Bash

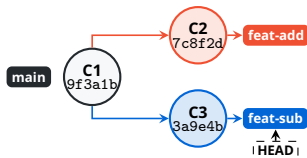
```
git switch -c feat-add  
# Ajout de add()  
git commit -am "feat: ajout add"
```

Fichier `math.h` (commit C2)

```
#ifndef MATH_H  
#define MATH_H  
int add(int a, int b) {return a+b;}  
#endif
```

Exemple de conflit (3/7)

Nouvelle branche `feat-sub` et implémentation de la soustraction



Commandes Bash

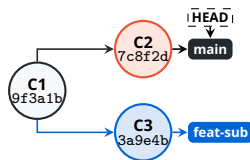
```
git switch main
git switch -c feat-sub
# Ajout de sub()
git commit -am "feat: ajout sub"
```

Fichier `math.h` (commit C3)

```
#ifndef MATH_H
#define MATH_H
int sub(int a, int b) {return a-b;}
#endif
```

Exemple de conflit (4/7)

Fusion Fast-Forward sans conflit



Fast-Forward instantané

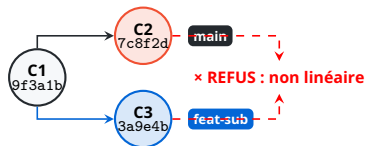
```
git switch main
git merge --ff-only feat-add
# Pas de problème
```

Fichier `math.h` sur `main`

```
#ifndef MATH_H
#define MATH_H
int add(int a, int b) {return a+b;}
#endif
```

Exemple de conflit (5/7)

CONFLIT!!! :(



Rejet par Git

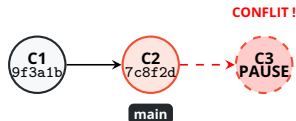
```
git switch main
git merge --ff-only feat-sub
# fatal: Not possible to fast-
# forward, aborting.
```

Sécurité

- main a divergé (est passé sur C2, pas C1)
- Git ne sais pas quel code conserver donc bloque
- -ff-only : Aucune balise de conflit injectée

Exemple de conflit (6/7)

Gestion du conflit



Rebase interrompu par Git

```
git switch feat-sub
git rebase main
# CONFLICT: Merge conflict in math.h
# Resolve all conflicts manually,
# mark them as resolved with
# "git add <paths>", then run
# "git rebase --continue".
```

Fichier `math.h` en cours de rebase

```
#ifndef MATH_H
#define MATH_H
<<<< HEAD
int add(int a, int b) {return a+b;}
||||| parent of 5f2a084 (feat: ajout sub)
// Fonctions maths a venir ...
=====
int sub(int a, int b) {return a-b;}
>>>> 5f2a084 (feat: ajout sub)
#endif
```

Exemple de conflit (7/7)

Résolution



Résolution et Fast-Forward final

```
# On gère manuellement le conflit
git add math.h
git rebase --continue
git switch main
git merge --ff-only feat-sub
# Succes !
```

Fichier final propre math.h

```
#ifndef MATH_H
#define MATH_H
int add(int a, int b) {return a+b;}
int sub(int a, int b) {return a-b;}
#endif
```

Installer et configurer Git

- **Installation sur votre système :**
 - **Debian** : `sudo apt install git`
 - **Arch** : `sudo pacman -S git`
 - **Nix** : `nix shell nixpkgs#git`

Configuration initiale obligatoire (signature des commits)

```
# Définir votre identité visible dans l'historique
git config --global user.name "Ada Lovelace"
git config --global user.email "ada.lovelace@example.com"

# Nommer la branche principale par défaut
git config --global init.defaultBranch main
```

Config de merge / pull

Configuration de git merge :

1. Mode standard (-ff par défaut) :

```
git config --global merge.ff true
```

2. Forcer un commit de fusion (-no-ff) :

```
git config --global merge.ff false
```

3. Strictement linéaire (-ff-only) :

```
git config --global merge.ff only
```

Configuration de git pull :

Évite les commits de merge surprises (recommandé) :

```
git config --global pull.ff only
```

Alternative : rebase automatique lors d'un pull :

```
git config --global pull.rebase true
```

Vérifier la configuration active :

```
git config --get merge.ff  
git config --get pull.ff
```

Configuration SSH

S'authentifier de façon moderne et sans mot de passe

1 Générer une paire de clés Ed25519 (plus rapide et sécurisée que RSA) :

```
ssh-keygen -t ed25519 -C "prenom.nom@example.com"
# Taper Entrée pour valider l'emplacement par défaut (~/.ssh/id_ed25519)
```

2 Afficher et copier sa clé publique :

```
cat ~/.ssh/id_ed25519.pub
# Copier la chaîne commençant par ssh-ed25519 AAAAC3NzaC1l...
```

3 Ajouter la clé sur GitHub / GitLab :

- Aller dans *Settings* → *SSH and GPG keys* → *New SSH Key*.

4 Tester la connexion :

```
ssh -T git@github.com
# Message attendu : Hi username! You've successfully authenticated...
```

Exercice : Gestion des conflits

Objectif

Insérer votre fragment dans README.md dans la partie correspondante

Comment faire?

```
git clone <URL_DU_DEPOT>
git switch -c feat-partie-X
# Coller votre texte dans README.md
git commit -am "feat: partie X"
git push origin main
```

`git push -force` **STRICTEMENT INTERDIT**

En cas de conflit

```
git pull origin main
```

Gardez les 2 textes et effacez les 3 lignes de balises

```
<<<<<< HEAD
=====
>>>>>>
```

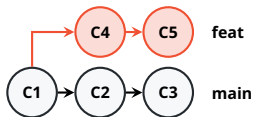
Puis finalisez :

```
git add README.md
git commit -m "fix: fusion partie X"
git push origin main
```

Rebase

Déplacer une branche au sommet d'une autre

Avant rebase :



Après git rebase main :



Commande

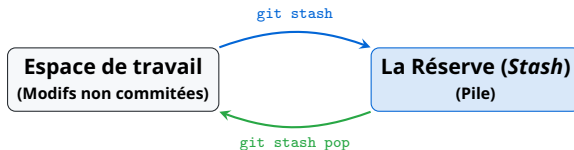
```
git switch feat  
git rebase main
```

En résumé

- Rejoue les commits locaux au bout de main
- Historique linéaire

Le Stash

Mettre son travail de côté sans faire de commit



Commandes essentielles

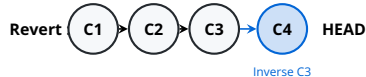
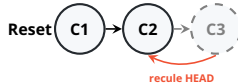
```
git stash      # Sauvegarde
git stash pop  # Restaure
git stash list # Affiche la pile
```

Cas d'usage

- Changer de branche en urgence
- Faire un pull sur un dossier non propre
- Tester une idée sans polluer l'historique

Reset vs Revert : Annuler des modifications

Réécrire le passé (local) ou avancer pour corriger (public)



git reset (Local uniquement)

Reculer le pointeur HEAD :

- -soft : garde le code indexé
- -hard : **détruit définitivement**

```
git reset --soft HEAD~1
```

git revert (Public & Collaboratif)

Crée un commit qui applique l'inverse :

- Historique passé intact
- 100% sécurisé sur main

```
git revert <commit-sha>
```

Conclusion

- **Commits atomiques** : une tâche = un commit avec un message descriptif.
- **Ne jamais commiter de secrets** : mots de passe, tokens et clés vont dans `.gitignore`.
- **Toujours faire un** `git pull` avant de débiter une session de travail.
- **Travailler sur des branches isolées**, jamais directement sur `main`.
- **Préférer** `revert` à `reset` sur toute branche déjà partagée en ligne.

Merci pour votre attention !

 Formation Robotech — Licence CC BY-NC-SA 4.0